T1 分段异或（xor4）

用阶段表示操作区间

本题要同时选择操作区间和求和子段。两者可以不同：某些操作可以完全位于求和子段之外，但恰好 m 次操作仍然必须完成。

先看 $m=0$。以 i 结尾的最大子段，要么只取当前位置，要么接在以 $i-1$ 结尾的最优子段之后。加入操作后，这个分类仍然成立，只是还要知道当前位置用了哪个掩码。

操作区间按顺序出现，所以不必记录各个端点，只记录当前位置所处的阶段：

- $j=0$：第一段操作之前；
- $j=2t-1$：第 t 段操作之内，当前位置变成 $v_i \operatorname{xor} p_t$；
- $j=2t$：第 t 段之后、第 $t+1$ 段之前，当前位置不变。

例如 $m=2$ 时，阶段依次为“第一段前、第一段内、两段之间、第二段内、第二段后”。相邻位置可以停留在同一阶段，也可以进入下一阶段。还允许从 1 直接进入 3，表示两段相邻，中间没有空隙；不能跳过操作段，否则这一段会变成空段。

同时维护最大子段和

令 $f_{i,j}$ 表示：前 i 个位置的操作安排合法，位置 i 处于阶段 j，求和子段非空且以 i 结尾时的最大和。后面尚未开始的操作可以留到后缀完成。记位置 i 在阶段 j 下的值为 w：偶数阶段取 v_i，奇数阶段取 $v_i \operatorname{xor} p_{(j+1)/2}$。

相同的 (i,j) 之后能做的选择完全相同，因此只保留其中最大的和。按 i 从小到大计算，每个状态只有两类来源。

从 i 重新开始子段。 此时令 $f_{i,j}=w$，但要保证子段之前放得下已经经过的操作：

- $j=0$ 时，任意 $i\ge 1$ 都可以；
- $j=2t-1$ 时，前面至少放 $t-1$ 个非空段，所以需要 $i\ge t$；
- $j=2t$ 时，前面至少放 t 个非空段，当前位置还要留作空隙，所以需要 $i\ge t+1$。

这些条件也足够：把前面的操作都放成单点即可。代码用 `first` 保存这一最小下标。

接在上一位置的子段之后。 上一阶段可以是 j、$j-1$；当 j 为不小于 3 的奇数时，还可以是 $j-2$，对应跳过两段之间的空隙。枚举这些合法前驱 q，执行

$$f_{i,j}=\max(f_{i,j},f_{i-1,q}+w).$$

初始 $f_{0,j}=-\infty$，每个新状态也先设为 $-\infty$。这样不会把空子段当成答案，也不会从不存在的状态转移。

何时可以结束子段？ 若当前位于第 t 段内或其后的空隙，后面还需放 $m-t$ 个非空操作段，每段至少一个位置。因此，只有

$$n-i\ge m-\left\lfloor\frac{j+1}{2}\right\rfloor$$

时，才能用 $f_{i,j}$ 更新答案。满足时，把剩余操作依次放成单点即可；不满足时，虽然求和子段已经选好，但整个操作方案无法完成。

任意合法方案的子段左端满足重开条件，中间的阶段变化都在上述转移中，右端满足结束条件，所以不会漏解。反过来，这三个条件保证每个计入答案的状态都能补成恰好 m 个非空操作段，所以不会算入非法方案。

每个位置有 $2m+1$ 个状态，每个状态至多检查三个前驱。时间复杂度为 $O(n(m+1))$，完整 DP 表的空间复杂度为 $O(n(m+1))$。最大约占 14 MiB，无须滚动数组。答案和 DP 用 `long long`，异或先在 `int` 上计算。

部分分思路

下表分值对应题面中的各组测试点，独立相加。这里只概述做法，不另附部分分代码。

条件	分值	做法
$m=0$	10 分	普通最大子段和，$O(n)$。
$n\le 14$	10 分	搜索每段左右端点，完成操作后扫描最大子段和。
$n\le 1000$	10 分	枚举求和子段左端点，再按操作阶段向右递推，$O(n^2(m+1))$。
$m=1$	10 分	只需维护操作前、操作中、操作后三个阶段，$O(n)$。
$m=2$	10 分	扩展为五个阶段，注意允许两段相邻，$O(n)$。
所有 $v_i\ge 0$	10 分	异或不改变符号位，最大子段可取全数组；只做操作阶段 DP，$O(n(m+1))$。
所有 $v_i<0$	10 分	操作后仍全负，最优子段只有一个位置；枚举它属于哪段操作，$O(n(m+1))$。

最后一档仍要检查可行性：位置 i 属于第 t 段，需要 $i\ge t$ 且 $n-i\ge m-t$；若想不修改它，需要其余 $n-1$ 个位置能放下 m 段，即 $n>m$。

满分代码

```
#include<bits/stdc++.h>
using namespace std;

const int N = 2e5 + 10;
const long long inf = 1LL << 60;
int n,m,p[5];
long long f[N][9];

int main(){
    ios::sync_with_stdio(false);
    cin.tie(0);
    cin >> n >> m;
    for(int i = 1; i <= m; i++){
        cin >> p[i];
    }
    for(int j = 0; j <= 2 * m; j++){
        f[0][j] = -inf;
    }
    long long ans = -inf;
    for(int i = 1; i <= n; i++){
        int x;
        cin >> x;
        for(int j = 0; j <= 2 * m; j++){
            long long w = x;
            if(j % 2 == 1) w = x ^ p[(j + 1) / 2];
            f[i][j] = -inf;

            // 子段从 i 开始，前面要能放下该阶段之前的操作。
            int first = 1;
            if(j > 0) first = j / 2 + 1;
            if(i >= first) f[i][j] = w;

            long long pre = f[i - 1][j];
            if(j >= 1) pre = max(pre,f[i - 1][j - 1]);
            // 两个操作段可以相邻，中间的空隙可以跳过。
            if(j >= 3 && j % 2 == 1) pre = max(pre,f[i - 1][j - 2]);
            if(pre != -inf) f[i][j] = max(f[i][j],pre + w);

            int need = m - (j + 1) / 2;
            if(n - i >= need) ans = max(ans,f[i][j]);
        }
    }
    cout << ans << '\n';
    return 0;
}
```

T2 三点分块（triplet）

先处理环外的树枝

若 n 不是 3 的倍数，直接无解。接下来考虑该删哪些边。

最直接的做法是枚举一条环边删去，把图变成树，再从叶子往上分块。但这样会重复处理相同的树枝。实际上，环外边的删留不受环上切法影响，可以先一次处理完。

将所有度数为 1 的点放入队列，逐层剥去叶子。用 `sz[x]` 表示当前还连在 x 上、尚未独立分成三点块的点数，初始为 1。处理 x 时，它只剩一条通向内部的边，设另一端为 y：

- `sz[x]` 为 1 或 2：这些点无法自己成块，必须保留边 xy，把点数加入 `sz[y]`。
- `sz[x]` 为 3：它们已经成块，必须删除边 xy，否则再加入 y 就超过三点。
- `sz[x]` 大于 3：无解。此前接入 x 的每个小块都不足三点，不能单独切走，而它们合在一起又太大。

每一步都没有选择，所以记下的环外删边在所有合法方案中相同。剥叶结束后，度数不为零的点恰好构成环；每个环点还带着若干个必须与它放在同一块中的树枝点。

环上为什么只剩三个候选

沿环依次记这些点数为 $s_1, s_2, \ldots, s_h$。若某个 $s_i > 3$，仍然无解。否则每个 s_i 都在 $1 \sim 3$ 之间。问题变成：把这个环切成若干连续段，使每段点数之和恰好为 3。

令

$$P_i = s_1 + s_2 + \cdots + s_i, \qquad S = P_h.$$

把第 i 个环点之后的边记作 e_i。已经分出去的块都是三点块，所以 S 仍是 3 的倍数。

先考虑至少删一条环边的方案。相邻两条删边之间的点数为 3，因此两条边对应的前缀和模 3 相同。沿环一周，**所有删边必定对应同一个余数 r**。

反过来，余数为 r 的环边也必须全部删除。否则，某个三点块内部还存在一个同余边界；它把这个块分成两个非空部分，而两部分的点数都能被 3 整除，总和至少为 6，矛盾。因此选定 r 后，删边方案就确定了，只需尝试 $r = 0, 1, 2$。

怎样判断一个余数是否合法？令 `cnt[r]` 为满足 $P_i \bmod 3 = r$ 的环边数。相邻两条这样的边之间，点数都是正的 3 的倍数，至少为 3；跨过环首尾的那段也成立，因为 S 是 3 的倍数。于是

$$\boxed{\text{余数 } r \text{ 对应的方案合法} \iff \mathrm{cnt}[r] = S/3.}$$

必要性：每块三点，共需要 $S/3$ 块。充分性：这些间隔的总点数为 $S = 3\,\mathrm{cnt}[r]$，每个间隔又至少有三点，因此它们都只能恰好有三点。

例如环上权值为 $1, 2, 1, 2$，前缀和为 $1, 3, 4, 6$。余数 0 出现两次、余数 1 也出现两次，均等于 $S/3 = 2$，所以得到两种合法切法。仅检查 S 能被三整除是不够的：权值为 $2, 2, 2$ 时，没有连续段的和能为 3，三个余数也都只出现一次。

保留三角形，再比较完整序列

上面还没有考虑“不删环边”。它合法当且仅当 $S = 3$。简单环至少有三个点，每个 $s_i \geq 1$，所以此时剩下的正好是一个三角形，可以整体保留。

我们至多得到四个候选。对每个候选，按原边号从小到大扫描，把强制删除的环外边和本次删除的环边一起加入序列，比较完整序列的字典序。

不能优先选删边少的方案。例如环外必须删除边 4 时，保留三角形得到 $[4]$；再删除环边 1 仍然合法，却得到更小的 $[1,4]$。但若保留三角形得到的序列是另一个候选的前缀，则保留方案更小。代码中的 `now < ans` 是两个 `vector` 的字典序比较，包含前缀和空序列的情况。

剥叶得到所有强制边，三个余数覆盖所有切环方案，再加上唯一可能的不切环方案，因此候选没有遗漏。每条边只处理常数次，至多构造四个长度为 $O(n)$ 的序列，时间、空间复杂度均为 $O(n)$。

部分分思路

下表各行分值独立相加，不是累计分。

条件	分值	做法
$n \le 18$	20 分	枚举删边集合，检查每块是否恰有三点，再比较删除序列，$O(n2^n)$。
$n \le 2000$	30 分	枚举一条环边删去，按树上的强制规则分块，$O(nh)$。
环长 $h \le 6$	10 分	同上，只需要重复至多六次树处理，$O(nh)$。
图为纯环	10 分	$n = 3$ 时保留三角形；否则每三个点切一块，只比较三种起点，$O(n)$。

枚举断环的做法还要尝试恢复最先删除的环边：若它的两端已经属于同一个三点块，恢复后仍合法，这样才不会漏掉保留三角形的方案。结合小规模、短环和纯环三种处理，可以覆盖题面前 70 分。

满分代码

```cpp
#include<bits/stdc++.h>
using namespace std;

const int N = 5e5 + 10;
struct node{
    int to,id;
};
vector<node> v[N];
vector<int> ans;
int n,deg[N],sz[N],tag[N],cnt[3];
bool cut[N],ok;

// r = -1 时不删环边，否则删去前缀和模 3 为 r 的环边。
void update(int r){
    vector<int> now;
    for(int i = 1; i <= n; i++){
        if(cut[i] || (r >= 0 && tag[i] == r)) now.push_back(i);
    }
    if(!ok || now < ans){
        ok = true;
        ans = now;
    }
}

int main(){
    ios::sync_with_stdio(false);
    cin.tie(0);
    cin >> n;
    for(int i = 1; i <= n; i++){
        int x,y;
        cin >> x >> y;
        v[x].push_back({y,i});
        v[y].push_back({x,i});
        deg[x]++;
        deg[y]++;
    }
```

```
    if(n % 3 != 0){
        cout << -1 << '\n';
        return 0;
    }
    queue<int> q;
    for(int i = 1; i <= n; i++){
        sz[i] = 1;
        tag[i] = -1;
        if(deg[i] == 1) q.push(i);
    }
    while(!q.empty()){
        int x = q.front();
        q.pop();
        if(sz[x] > 3){
            cout << -1 << '\n';
            return 0;
        }
        deg[x] = 0;
        for(int i = 0; i < (int)v[x].size(); i++){
            int y = v[x][i].to,id = v[x][i].id;
            if(deg[y] == 0) continue;
            if(sz[x] == 3) cut[id] = true;
            else sz[y] += sz[x];
            deg[y]--;
            if(deg[y] == 1) q.push(y);
        }
    }

    int start = 1;
    while(deg[start] == 0) start++;
    int x = start,pre = 0,sum = 0;
    do{
        if(sz[x] > 3){
            cout << -1 << '\n';
            return 0;
        }
        sum += sz[x];
        for(int i = 0; i < (int)v[x].size(); i++){
            int y = v[x][i].to,id = v[x][i].id;
            if(deg[y] == 0 || y == pre) continue;
            tag[id] = sum % 3;
            cnt[tag[id]]++;
            pre = x;
            x = y;
            break;
        }
    }while(x != start);

    if(sum == 3) update(-1);
    for(int r = 0; r < 3; r++){
        if(cnt[r] == sum / 3) update(r);
    }
    if(!ok){
        cout << -1 << '\n';
        return 0;
    }
    cout << ans.size() << '\n';
    for(int i = 0; i < (int)ans.size(); i++){
        if(i) cout << ' ';
        cout << ans[i];
    }
    cout << '\n';
    return 0;
}
```

T3 菜系点餐（menu）

最少周数由哪一批菜决定？

每道菜只需点一次，重复点不会帮助覆盖。先把每道菜交给一位接受它的人。若第 i 个人分到 b_i 道菜，每周点一道，这种分配就需要 $\max b_i$ 周。问题变成：怎样分配，才能使每人菜数的最大值最小？

只看总菜数和总人数是不够的。比如三个菜系各有两道菜，甲接受 A、B，乙和丙只接受 C。虽然六道菜、三个人平均只需两周，但 A、B 的四道菜都要交给甲，实际至少需要四周。只检查每个菜系，也发现不了这个限制。

因此，要检查**一组菜系共同占用的人手**。对菜系集合 S，记：

- $D(S)$：这些菜系的菜数之和。
- $P(S)$：至少接受其中一个菜系的人数。每个人只算一次。

若用 W 周完成，这些菜只能由这 $P(S)$ 个人来点，所以必须满足

$$D(S) \le WP(S).$$

这给出了周数的下界。还需要证明：所有集合都满足这个条件时，确实能够完成分配。

为什么这些条件足够？

把每个人拆成 W 个名额，分别代表他在各周能点的一道菜。每道菜连向所有接受它的人的名额。能否安排完全部菜，就是能否让每道菜匹配到不同的名额。

Hall 定理告诉我们：这样的匹配存在，当且仅当**任意一批菜能用的名额数，不少于这批菜的数量**。

任取一批具体的菜 X，设它们涉及的菜系集合为 S。同一菜系的所有菜都能交给同一批人，因此 X 一共能使用 $WP(S)$ 个名额。同时，X 只是这些菜系中的一部分菜，所以

$$|X| \le D(S) \le WP(S).$$

这就保证了任意 X 都满足 Hall 条件，全部菜一定能安排完。

因此，只要不存在 $D(S) > 0$ 而 $P(S) = 0$ 的集合，答案就是

$$\boxed{\max_{S:D(S)>0} \left\lceil \frac{D(S)}{P(S)} \right\rceil}.$$

比它小一定不够；取这个值时，所有集合的容量条件又都成立。这样就直接求出了答案，无需二分周数。有菜却无人接受时输出 -1；没有菜的集合不产生限制。

用子集和一次算出所有人数

$c \le 22$，可以用二进制表示菜系集合。最低位对应菜系 1；输入串从左到右第 j 个字符对应二进制第 $j-1$ 位。

先令 `f[S]` 表示接受集合**恰好为** S 的人数。逐个枚举菜系集合、再扫描所有人，需要 $O(n2^c)$，太慢。改为统计反面：一个人完全不接受 S，当且仅当他的接受集合包含于 S 的补集。

于是，只要把 `f[T]` 变成“接受集合包含于 T 的人数”，就有

$$P(S) = n - f[ALL \setminus S].$$

这里 ALL 是全部菜系。接受集合为空的人也必须计入 `f[0]`，因为他不能帮助任何非空菜系集合，应当始终被减掉。

怎样求每个集合的子集和？依次处理每个二进制位 i。当 S 含有这一位时，将不含它的对应状态加过来：

$$f[S] \mathrel{+}= f[S \setminus \{i\}].$$

处理前 k 位后，`f[S]` 统计的集合在前 k 位可以从 S 中删去元素，其他位仍与 S 相同。处理下一位时，原来的部分保留这一位，加来的部分删去这一位；两部分互不相交，因此不会重复。全部位处理完后，统计的恰好就是 S 的所有子集。

右侧不含当前位，本轮不会被修改，所以可以直接在原数组中更新。

菜量也用同一段循环计算：起初只令 `sum[1 << (j - 1)]` 等于菜系 j 的菜数，其他位置为零。做完相同的子集求和后，每个单元素集合恰好贡献一次，`sum[S]` 就是 $D(S)$。

最后枚举非空集合 S，用 `sum[S]` 和 `n - f[ALL ^ S]` 计算上取整的比值，取最大值。`sum[S]` 为零时跳过；菜量大于零但人数为零时无解。

读取输入需要 $O(m+nc)$，子集求和需要 $O(c2^c)$，最后枚举需要 $O(2^c)$。总时间复杂度为 $O(m+nc+c2^c)$，空间复杂度为 $O(2^c)$。两张整型数组约占 32 MiB，菜量和人数都能用 `int` 保存。

部分分思路

下表分值对应题面的独立点组，不表示这些算法的累计得分。

条件与分值	做法
$n \le 5, m \le 8, c \le 4$，20 分	搜索每道菜交给谁，记录每人的菜数，最小化最大值。最多 n^m 种分配。
$c \le 10$，20 分	枚举菜系集合，逐人检查是否接受其中某一菜系，求出 $D(S), P(S)$。时间 $O(m+nc+(n+c)2^c)$。
$n \le 18$，10 分	改为枚举人群 T。把每个菜系的菜数加到该菜系的接受者集合上，再按人群集合做子集和，得到只能交给 T 的菜数 $G(T)$。这些菜需要 $\lceil G(T)/\lvert T\rvert \rceil$ 周，取最大值；$G(\varnothing) > 0$ 时无解。时间 $O(m+nc+n2^n)$。
每人恰好接受一个菜系，10 分	各菜系的人手互不重叠，分别计算菜量除以人数的上取整，取最大值。有菜无人时无解。时间 $O(m+nc)$。
$c \le 18$，10 分	合并相同接受集合后，对每个集合直接枚举所有子集来求 `f`，再使用补集公式。每个元素在“集合外、集合内但不在子集、子集内”三种情况中选一种，总计 $O(3^c)$ 次。总时间 $O(m+nc+3^c+c2^c)$。

人数子集做法的充分性与前面相同：对任意一批菜，令 T 为能接受其中某道菜的全部人。这批菜都被计入 $G(T)$，所以 $G(T) \le W\lvert T\rvert$ 就保证了这批菜的 Hall 条件。

满分代码

```
#include<bits/stdc++.h>
using namespace std;

const int N = (1 << 22) + 10;
int n,m,c,f[N],sum[N],ans;

int main(){
    ios::sync_with_stdio(false);
    cin.tie(0);
    cin >> n >> m >> c;
    for(int i = 1; i <= m; i++){
```

```
        int x;
        cin >> x;
        sum[1 << (x - 1)]++;
    }
    for(int i = 1; i <= n; i++){
        string s;
        cin >> s;
        int x = 0;
        for(int j = 0; j < c; j++){
            if(s[j] == '1') x |= 1 << j;
        }
        f[x]++;
    }
    int all = (1 << c) - 1;
    for(int i = 0; i < c; i++){
        for(int s = 0; s <= all; s++){
            if(s & (1 << i)){
                f[s] += f[s ^ (1 << i)];
                sum[s] += sum[s ^ (1 << i)];
            }
        }
    }
    for(int s = 1; s <= all; s++){
        if(sum[s] == 0) continue;
        int cnt = n - f[all ^ s];
        if(cnt == 0){
            cout << -1 << '\n';
            return 0;
        }
        ans = max(ans,(sum[s] + cnt - 1) / cnt);
    }
    cout << ans << '\n';
    return 0;
}
```

T4 丢失的异或方程（logic）

给出 K 个不同的赋值，要求恢复一个异或方程组，解集恰好是这些赋值。困难在于：让输入都满足方程还不够，还要保证方程不会产生其他解。

先把第一组赋值变成全零

设第一组赋值为 s_1，将每组赋值变成差分

$$v_i = s_i \oplus s_1.$$

假设一条方程选中了若干变量，它在 s_i 和 s_1 上的结果都是 b。把两式异或，右端变成 $b \oplus b = 0$，左端就是这些变量在 v_i 中的异或。因此，**原方程的右端虽然可能是 1，但所有差分都满足右端为 0 的方程。**

进一步，两个满足齐次方程的差分异或后，仍然满足方程。所以任何几个输入差分的异或，也必须是允许的差分。如果这样产生了输入中没有的向量，原方程组就一定会多出解。

用线性基判断是否多出解

把所有差分加入异或线性基，设留下 r 个线性无关的向量。它们所有可能的异或组合构成集合 V。

每个基向量都可以选或不选，共有 2^r 种组合。这些组合互不相同：如果两种不同选法得到同一个结果，把它们异或，就会得到一组非空基向量的异或为零，与线性无关矛盾。

输入中的 K 个差分互不相同，而且都属于 V，于是：

- 若 $K < 2^r$，V 中一定有未给出的差分，无法构造；
- 若 $K = 2^r$，输入恰好列出了 V，接下来只需构造能描述 V 的约束。

因此判断条件就是 $K = 2^r$。不能只判断 K 是否是二的幂：例如 `000,100,010,001` 有四个赋值，却能生成八个差分。

实现时，`b[i]` 保存第一个 `1` 在第 i 列的基向量。插入一个差分，从左到右检查每一列：这一列为 `0` 就跳过；已有 `b[i]` 就异或它，消掉这一位；否则把当前向量存入 `b[i]`，基的大小增加一。

从差分恢复方程

现在要选出一组系数，使每个差分在这些位置上的异或都是零。只需对所有基向量成立，就能对它们的任意异或组合成立。

先从右到左处理线性基，消掉其他行在主元列上的 `1`，使每个主元列只剩所在行的一个 `1`。其余列称为自由列。

对每个自由列 j，构造一条方程的系数：

1. 第 j 列设为 1，其他自由列设为 0。
2. 对每个主元列 i，第 i 列设为基向量 `b[i]` 在第 j 列的值。

为什么这样选？在基向量 `b[i]` 中，只有它自己的主元列可能参与，其他主元列都是零；自由列中也只有第 j 列参与。因此选中位置的异或为

$$b[i]_j \oplus b[i]_j = 0.$$

这样得到的系数对所有差分都成立。最后，把这条方程的右端设为第一组赋值 s_1 在选中位置上的异或，便平移回了原问题。

例如差分基为 `101,011`，第三列是自由列。构造出的系数为 `111`；若 $s_1 = 001$，就得到 $x_1 \oplus x_2 \oplus x_3 = 1$。

自由列有 $n - r$ 个，构造出的方程也有 $n - r$ 条。它们在自由列上的系数各有一个独立的 `1`，所以彼此独立。$n - r$ 条独立方程消去后有 r 个变量可以自由取值，恰好有 $2^r = K$ 个解。我们已经保证输入都是解，因此没有多余解。

这也证明了条数最少：少一条方程，独立约束最多只有 $n - r - 1$ 条，至少留下 $r + 1$ 个自由变量，解就会多于 K 个。

再消元一次，得到规范输出

刚才规范化的是差分基，构造出的方程不一定已经规范。例如差分基只有 `111` 时，两条系数为 `110,101`，它们的第一个 `1` 在同一列，不能直接输出。

对方程系数矩阵从左到右消元：在当前列寻找主元行，交换到下一行，再把其他所有行的这一列消成零。每次异或系数行时，右端也要一起异或。这样主元列递增，且每个主元列只有一个 `1`，正好满足题目要求。

代码用 `long long` 存差分基，第 j 个字符对应二进制第 $j - 1$ 位；方程直接存入二维数组 `a`，右端另存 `rhs`。读入及线性基插入为 $O(Kn)$，最后的普通矩阵消元为 $O(n^3)$，总时间 $O(Kn + n^3)$，空间 $O(n^2)$。这里 $n \le 50$，无需压缩方程矩阵。

当 $K = 1$ 时，差分基为空，会构造 n 条固定变量的方程；当 $r = n$ 时，无需任何方程，输出 `0`。

部分分思路

以下分值分别对应题面中的独立点组。

- **20 分，$n \le 10$：** 枚举所有非零系数，右端由第一组赋值确定，再检查其余赋值是否满足。用线性基保留其中的独立方程，再整理为规范输出，设有 q 条独立方程；只有 $2^{n-q} = K$ 时才合法。时间 $O(nK2^n + n^3)$。
- **30 分，$K \le 256$：** 将差分排序，枚举任意两个差分，二分检查它们的异或是否仍在集合中。差分集合含零；若对异或封闭，就包含任意多个差分的异或，可以用上面的办法恢复方程。时间 $O(Kn + K^2 \log K + n^3)$。
- **10 分，特殊性质 A：** 扫描每一列，只出现一个值的变量就是固定变量，直接输出对应的单变量方程。其余变量可以独立取值是本档的保证，一般数据不能这样处理。时间 $O(Kn + n^2)$。

满分代码

```
#include<bits/stdc++.h>
using namespace std;

const int N = 55;
int n,k,r,a[N][N],rhs[N];
long long b[N];

// b[i] 是第一个 1 在第 i 列的差分基向量。
void add(long long x){
    for(int i = 1; i <= n; i++){
        if(((x >> (i - 1)) & 1) == 0) continue;
        if(b[i] != 0){
```

```
            x ^= b[i];
        }else{
            b[i] = x;
            r++;
            return;
        }
    }
}

int main(){
    ios::sync_with_stdio(false);
    cin.tie(0);
    cin >> n >> k;
    long long first = 0;
    for(int i = 1; i <= k; i++){
        string s;
        cin >> s;
        long long x = 0;
        for(int j = 1; j <= n; j++){
            if(s[j - 1] == '1') x |= 1LL << (j - 1);
        }
        if(i == 1) first = x;
        add(x ^ first);
    }
    if((1LL << r) != k){
        cout << -1 << '\n';
        return 0;
    }

    // 消掉其他基向量中的主元，使每个主元列只有一个 1。
    for(int i = n; i >= 1; i--){
        if(b[i] == 0) continue;
        for(int j = 1; j < i; j++){
            if((b[j] >> (i - 1)) & 1) b[j] ^= b[i];
        }
    }

    int q = 0;
    for(int j = 1; j <= n; j++){
        if(b[j] != 0) continue;
        q++;
        a[q][j] = 1;
        for(int i = 1; i <= n; i++){
            if(b[i] != 0) a[q][i] = (b[i] >> (j - 1)) & 1;
        }
        // 右端由第一组赋值确定。
        for(int i = 1; i <= n; i++){
            if(a[q][i] == 1 && ((first >> (i - 1)) & 1)) rhs[q] ^= 1;
        }
    }

    // 对构造出的方程组再消元，满足题目要求的输出顺序。
    int row = 0;
    for(int j = 1; j <= n; j++){
        int p = row + 1;
        while(p <= q && a[p][j] == 0) p++;
        if(p > q) continue;
        row++;
        for(int t = 1; t <= n; t++) swap(a[row][t],a[p][t]);
        swap(rhs[row],rhs[p]);
        for(int i = 1; i <= q; i++){
            if(i == row || a[i][j] == 0) continue;
            for(int t = 1; t <= n; t++) a[i][t] ^= a[row][t];
            rhs[i] ^= rhs[row];
        }
    }

    cout << q << '\n';
    for(int i = 1; i <= q; i++){
        for(int j = 1; j <= n; j++) cout << a[i][j];
        cout << ' ' << rhs[i] << '\n';
    }
    return 0;
}
```